

Consistent Hashing

Phase 2 · Intermediate · Estimated time: 1.5 hours

1. Concept From First Principles

Chapter 6 mentioned that resharding is painful, and used `hash(user_id) % 4` as a simple example of a shard-routing function. This chapter explains exactly why that simple modulo approach is a trap at scale, and introduces the technique — consistent hashing — that most real distributed systems use instead, for sharding, for distributed caching, and for load balancing alike.

The problem with `hash(key) % N` (where N is the number of shards or cache servers) is what happens when N changes. Adding or removing even a single server changes N , which changes the result of the modulo operation for *almost every key* in the system — not just the keys that logically should move to or from the new server. Going from 4 servers to 5, for example, remaps roughly 80% of all keys to a different server, even though only 20% of keys actually needed to move to make room for the new server. In a distributed cache, this means a single server addition or removal can cause a massive, sudden wave of cache misses across almost the entire dataset — exactly the cache stampede problem Phase 1 warned about, self-inflicted by your own scaling operation.

Consistent hashing solves this by imagining a hash space as a fixed **ring** (a circle of hash values from 0 to some maximum, wrapping back to 0). Both the servers and the data keys are hashed onto points on this same ring. Each key is assigned to the first server encountered walking clockwise around the ring from the key's position. When a server is added, it only takes over the small arc of keys between itself and the next server clockwise — every other key's assignment is completely unaffected. When a server is removed, only its arc of keys moves to the next server clockwise; again, everything else stays put. This is the entire point: adding or removing one node only remaps roughly $1/N$ of keys, not almost all of them.

Backend engineer analogy

If your Node.js app uses Redis Cluster or a client-side consistent-hashing Redis client (like a ketama-based library), you've already relied on this exact mechanism without necessarily naming it: adding a new Redis node to the cluster doesn't invalidate your entire cache, only the small slice of keys that now belong to the new node. Without consistent hashing, scaling your cache cluster would mean briefly nuking almost your entire cache — a very expensive way to add capacity.

Virtual nodes: fixing uneven distribution

A naive ring with one point per physical server has a real weakness: with only a handful of servers, their random hash positions can land unevenly around the ring, giving some servers much larger arcs (and therefore much more load) than others purely by chance. The standard fix is **virtual nodes**: instead of hashing each physical server to one point on the ring, hash it to many points (commonly 100–several hundred), scattered around the ring. Each physical server now owns many small, scattered arcs instead of one large contiguous one, which averages out to a much more even overall load distribution, and also means that when one server fails, its load is spread across many other servers instead of

dumping entirely onto whichever single server happened to be its clockwise neighbor.

Consistent hashing shows up in more places than database sharding. Distributed caches (Memcached clients, Redis Cluster) use it to decide which cache node owns a given key. Content delivery networks and load balancers use it to route a given client or session consistently to the same backend server, which matters for sticky sessions (Phase 1 Chapter 9). Distributed hash tables in peer-to-peer systems use it as their foundational routing mechanism. The common thread in every case: you want to add or remove nodes without triggering a system-wide reshuffle of data or traffic.

It's worth being precise about what problem consistent hashing does *not* solve: it doesn't eliminate the need to actually move the affected slice of data when a node joins or leaves — some data movement is unavoidable and correct. What it eliminates is the *unnecessary* movement that plain modulo hashing causes. It's also worth noting that consistent hashing alone doesn't guarantee your shard key is well-chosen in the first place — Chapter 6's hotspot warning about a skewed key still applies even on a perfectly implemented consistent-hashing ring.

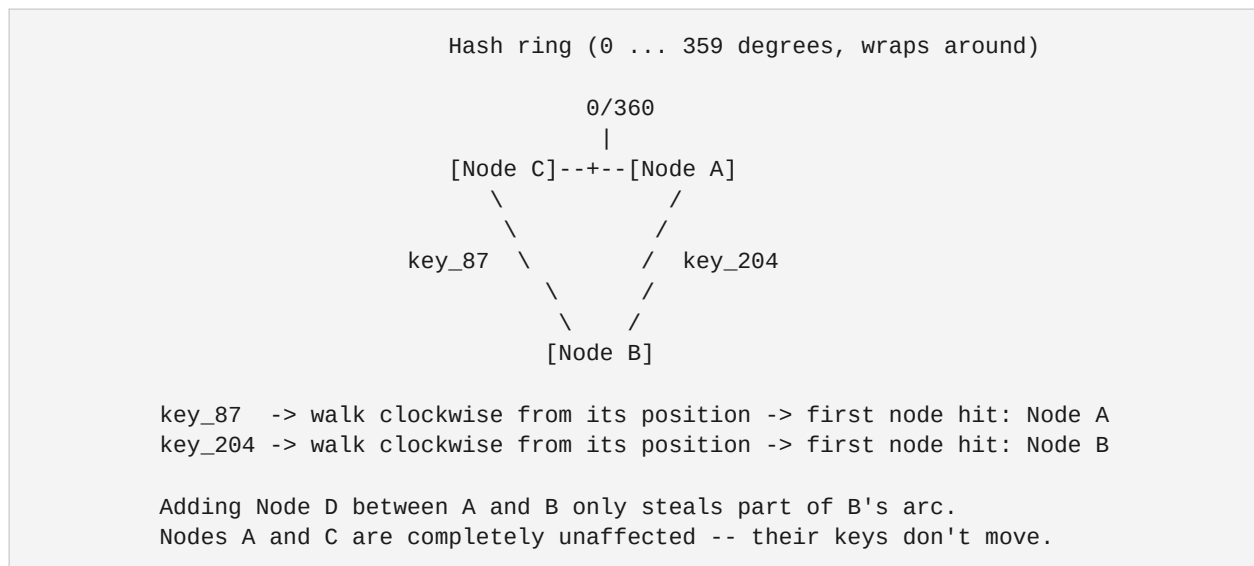
2. Real-World Example / Case Study

Distributed caching layers at Flipkart- and Swiggy-scale systems, sitting in front of product catalogs and restaurant menus, rely on consistent hashing so that scaling the cache cluster up during a flash sale or peak lunch rush doesn't itself trigger a self-inflicted cache stampede by invalidating most of the existing cached data the moment new cache nodes join.

Content delivery networks serving Careem, Talabat, and Noon route user requests to a consistent, geographically-appropriate edge server using hash-ring-style logic, ensuring that a given user (or a given piece of content) is consistently routed to the same nearby cache node across requests, which improves both cache hit rates and, for session-sensitive traffic, keeps a user's requests landing on a server that already has their session warmed.

Cassandra, used by several large-scale systems for high-write-throughput data (activity feeds, event logs, IoT-style telemetry), uses consistent hashing as its core data-distribution mechanism — it's one of the clearest, most textbook production implementations of everything this chapter describes, virtual nodes included.

3. Diagram



4. Trade-offs Table

Approach	Keys remapped on scale event	Complexity
Plain $\text{hash}(\text{key}) \% N$	$\sim(N-1)/N$ of all keys (almost everything)	Very simple to implement
Consistent hashing, no virtual nodes	$\sim 1/N$ of keys, but uneven load possible	Moderate: ring + binary search for placement
Consistent hashing + virtual nodes	$\sim 1/N$ of keys, evenly distributed	Slightly more bookkeeping, but standard in production libraries
Static, manual key-to-shard mapping	Zero, but doesn't scale operationally	Simple for tiny, fixed-size clusters only

5. Common Interview Questions

Q1: "Why not just use $\text{hash}(\text{key}) \% N$ to distribute keys across servers?"

Model approach: Walk through the concrete math — adding or removing one server changes N , which remaps almost every key's assignment, not just the keys that needed to move, causing a massive, avoidable wave of cache misses or data movement on every scaling event.

Q2: "Explain consistent hashing and why it fixes the modulo problem."

Model approach: Describe the ring, hashing both keys and nodes onto it, and the clockwise-nearest-node assignment rule, then explicitly state the guarantee: adding or removing one node only remaps that node's arc of keys, roughly $1/N$ of the total, leaving everything else untouched.

Q3: "What problem do virtual nodes solve, and how?"

Model approach: Explain that a small number of physical servers can land unevenly on the ring by chance, creating unbalanced load, and that giving each physical server many scattered virtual points averages this out to a much more even distribution, while also spreading a failed node's load across many survivors instead of dumping it on one neighbor.

Q4: "Where else, besides database sharding, is consistent hashing used?"

Model approach: Name at least two concrete examples — distributed caching (Redis Cluster, Memcached client-side hashing) and CDN/load-balancer request routing — and explain the common thread: any system that needs to add/remove nodes without a full data or traffic reshuffle.

6. Practice Exercises

- 1 With 4 cache servers and plain modulo hashing, calculate roughly what percentage of keys get remapped when a 5th server is added. Compare this to consistent hashing.
- 2 Explain, in your own words, why virtual nodes are necessary even though the basic consistent-hashing ring already solves the 'too much remapping' problem.
- 3 Design a scenario (in a food delivery app) where consistent hashing would be the right tool for a caching layer, and describe what happens on the ring when one cache node crashes.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design: (P-5): Consistent Hashing Explained Scalable and Fault-Tolerant System Design"	Solid full-length walkthrough of the ring mechanism.
English	"Consistent Hashing Explained: Interview Guide for Distributed Systems Mastery"	Interview-focused framing with common follow-up questions.
Hindi	"6. Consistent Hashing in Hindi with Example System Design"	Hindi explanation with a worked example on the ring.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers consistent hashing as part of a full bilingual system design course.

8. Key Takeaways

- Plain $\text{hash}(\text{key}) \% N$ remaps almost all keys whenever N changes, causing unnecessary mass data movement.
- Consistent hashing places both nodes and keys on a ring; a key belongs to the next node clockwise.
- Adding or removing a node only remaps that node's arc of keys, roughly $1/N$ of the total, not everything.
- Virtual nodes (many ring points per physical server) fix uneven load distribution from unlucky hash placement.
- Consistent hashing is used for sharding, distributed caching, CDN routing, and load balancer request routing.
- It doesn't replace good shard-key design — a skewed key can still create hotspots even on a perfect ring.
- Some data movement on scale events is unavoidable and correct; consistent hashing eliminates only the unnecessary excess.